

100 задач на строки в Go

1. Напишите функцию, которая возвращает количество байт и количество Unicode-символов в строке. Строка может содержать кириллицу, emoji и комбинируемые символы.

```
func StringSize(s string) (bytes int, runes int)
```

2. Напишите функцию, которая проверяет, состоит ли строка только из ASCII-символов.

```
func IsASCII(s string) bool
```

3. Напишите функцию, которая возвращает первый Unicode-символ строки и признак его наличия. Пустую строку обработайте без panic.

```
func FirstRune(s string) (rune, bool)
```

4. Напишите функцию, которая возвращает последний Unicode-символ строки и признак его наличия. Не считайте, что один символ занимает один байт.

```
func LastRune(s string) (rune, bool)
```

5. Разверните строку по Unicode-символам. Результат для корректной UTF-8 строки тоже должен быть корректной UTF-8 строкой.

```
func Reverse(s string) string
```

6. Удалите из строки все вхождения заданного Unicode-символа.

```
func RemoveRune(s string, target rune) string
```

7. Замените все вхождения одного Unicode-символа другим, не используя strings.ReplaceAll.

```
func ReplaceRune(s string, old, new rune) string
```

8. Проверьте, является ли строка палиндромом по Unicode-символам. Регистр и пробелы пока учитываются.

```
func IsPalindrome(s string) bool
```

9. Удалите пробельные символы в начале и конце строки без использования strings.TrimSpace. Учитывайте Unicode-пробелы.

```
func TrimUnicodeSpace(s string) string
```

10. Замените любую последовательность Unicode-пробелов одним обычным пробелом и удалите пробелы по краям.

```
func NormalizeSpaces(s string) string
```

11. Реализуйте проверку наличия подстроки без `strings.Contains` и `strings.Index`. Пустая подстрока считается найденной.

```
func Contains(s, sub string) bool
```

12. Верните байтовый индекс первого вхождения подстроки без `strings.Index`. Если вхождения нет, верните `-1`.

```
func IndexOf(s, sub string) int
```

13. Подсчитайте непересекающиеся вхождения подстроки. Для пустой подстроки верните ошибку.

```
func CountOccurrences(s, sub string) (int, error)
```

14. Подсчитайте все вхождения подстроки, включая пересекающиеся. Например, в "aaaa" строка "aa" встречается три раза.

```
func CountOverlapping(s, sub string) (int, error)
```

15. Найдите общий префикс двух строк по Unicode-символам.

```
func CommonPrefix(a, b string) string
```

16. Найдите самый длинный общий префикс всех строк в слайсе. Для пустого слайса верните пустую строку.

```
func LongestCommonPrefix(items []string) string
```

17. Разбейте строку по непустому разделителю без `strings.Split`. Сохраните пустые элементы между соседними разделителями и по краям.

```
func Split(s, sep string) ([]string, error)
```

18. Соедините строки разделителем без `strings.Join`. Реализация не должна иметь квадратичную сложность из-за многократной конкатенации.

```
func Join(parts []string, sep string) string
```

19. Удалите с обоих краёв все Unicode-символы, входящие в набор `cutset`, без `strings.Trim`.

```
func TrimSet(s, cutset string) string
```

20. Сожмите каждую подряд идущую группу одинаковых Unicode-символов до одного символа: "aaaabbv" -> "abv".

```
func CollapseRuns(s string) string
```

21. Реализуйте RLE-сжатие по Unicode-символам. Формат результата: количество, двоеточие, символ; группы разделяются точкой с запятой. Формат должен однозначно работать и для цифр, и для знаков пунктуации.

```
func EncodeRLE(s string) string
```

22. Реализуйте декодирование формата из предыдущей задачи. Отклоняйте повреждённые записи, нулевые количества и переполнение размера результата.

```
func DecodeRLE(encoded string) (string, error)
```

23. Подсчитайте слова, считая разделителями любые последовательности Unicode-пробелов.

```
func WordCount(s string) int
```

24. Постройте частотный словарь: приведите слова к нижнему регистру, отделите пунктуацию и подсчитайте каждое слово. Слова могут содержать кириллицу.

```
func WordFrequency(s string) map[string]int
```

25. Найдите первый Unicode-символ, который встречается в строке ровно один раз. Порядок определяется исходной строкой.

```
func FirstUniqueRune(s string) (rune, bool)
```

26. Найдите первый Unicode-символ, второе появление которого встречается раньше вторых появлений остальных символов.

```
func FirstRepeatedRune(s string) (rune, bool)
```

27. Проверьте, что все Unicode-символы строки уникальны.

```
func AllRunesUnique(s string) bool
```

28. Проверьте, являются ли две строки анаграммами. Игнорируйте регистр, Unicode-пробелы и пунктуацию.

```
func AreAnagrams(a, b string) bool
```

29. Сгруппируйте строки-анаграммы. Внутри группы сохраните исходный порядок, а группы расположите по первому появлению их элемента во входе.

```
func GroupAnagrams(words []string) [][]string
```

30. Верните самое длинное слово по числу Unicode-символов. При равенстве выберите первое. Если слов нет, верните false.

```
func LongestWord(s string) (word string, ok bool)
```

31. Разверните порядок слов, сведя любые разделяющие пробелы к одному: " один два три " -> "три два один".

```
func ReverseWords(s string) string
```

32. Разверните символы внутри каждого слова, но сохраните все исходные пробельные последовательности на своих местах.

```
func ReverseEachWord(s string) string
```

33. Сделайте первую букву каждого слова заглавной, остальные символы не меняйте. Границы слов определяйте по Unicode-пробелам.

```
func TitleWords(s string) string
```

34. Соберите инициалы из непустых слов: первый Unicode-символ каждого слова в верхнем регистре, между инициалами точки.

```
func Initials(fullName string) string
```

35. Преобразуйте camelCase, PascalCase, kebab-case и строки с пробелами в snake_case. Учтите границы аббревиатур: "HTTPServerID" -> "http_server_id".

```
func ToSnakeCase(s string) string
```

36. Преобразуйте snake_case, kebab-case и строки с пробелами в lowerCamelCase. Несколько разделителей подряд не должны создавать пустые слова.

```
func ToLowerCamel(s string) string
```

37. Нормализуйте строку в kebab-case: только строчные буквы и цифры, между словами один дефис, дефисов по краям нет.

```
func ToKebabCase(s string) string
```

38. Проверьте идентификатор: первый символ - Unicode-буква или подчёркивание, остальные - Unicode-буквы, цифры или подчёркивания.

```
func IsIdentifier(s string) bool
```

39. Провалидируйте пароль: не менее 12 Unicode-символов, есть строчная и заглавная буква, цифра и знак пунктуации, нет пробелов и трёх одинаковых символов подряд. Верните список нарушенных правил.

```
func ValidatePassword(s string) []string
```

40. Провалидируйте упрощённый email без regexp: ровно один @, непустые local и domain, допустимые символы, нет точек подряд, домен содержит точку, каждая метка домена корректна.

```
func ValidateEmail(s string) error
```

41. Распарсите десятичное целое число без strconv.Atoi. Разрешите необязательный знак, запретите пробелы и корректно обнаруживайте переполнение int64.

```
func ParseInt64(s string) (int64, error)
```

42. Распарсите беззнаковое число в системе счисления от 2 до 36 без strconv.ParseUint. Поддержите цифры 0-9 и буквы a-z/A-Z, проверяйте переполнение uint64.

```
func ParseUintBase(s string, base int) (uint64, error)
```

43. Преобразуйте `int64` в строку в системе счисления от 2 до 36 без `strconv.FormatInt`. Корректно обработайте `math.MinInt64`.

```
func FormatIntBase(n int64, base int) (string, error)
```

44. Распарсите десятичное число вида `[ -+ ]digits[.digits]` без `strconv.ParseFloat`. Экспонента не поддерживается; некорректный формат и переполнение должны давать ошибку.

```
func ParseDecimal(s string) (float64, error)
```

45. Распарсите логическое значение без `strconv.ParseBool`. Примите `true/false`, `yes/no`, `on/off` и `1/0` в любом регистре, остальные значения отклоните.

```
func ParseBool(s string) (bool, error)
```

46. Разберите строку `key=value`, где пары разделены точкой с запятой. Пробелы вокруг ключей и значений игнорируются, пустые ключи и дубликаты запрещены.

```
func ParseKeyValue(s string) (map[string]string, error)
```

47. Разберите одну CSV-строку: запятые внутри кавычек не разделяют поля, двойная кавычка внутри `quoted`-поля кодируется как две кавычки. Отклоняйте незакрытые кавычки и мусор после закрывающей кавычки.

```
func ParseCSVLine(line string) ([]string, error)
```

48. Сериализуйте поля в одну корректную CSV-строку. Добавляйте кавычки только когда нужно; корректно экранируйте кавычки, запятые и переводы строк.

```
func EncodeCSVLine(fields []string) string
```

49. Реализуйте percent-decoding для `query`-компонента URL: `%XX` декодируется в байт, + превращается в пробел. Отклоняйте неполные и не-hex escape-последовательности.

```
func QueryUnescape(s string) (string, error)
```

50. Реализуйте percent-encoding для `query`-компонента URL по байтам UTF-8. Неразрешённые байты кодируйте заглавными hex-цифрами, пробел кодируйте как +.

```
func QueryEscape(s string) string
```

51. Разберите `query string` в `map[string][]string`. Сохраняйте повторяющиеся ключи и пустые значения, применяйте percent-decoding, при первой ошибке верните её позицию.

```
func ParseQuery(raw string) (map[string][]string, error)
```

52. Нормализуйте URL-путь: схлопните повторные /, удалите сегменты `..`, обработайте `..` без выхода выше корня, сохраните признак завершающего слеша.

```
func CleanPath(path string) string
```

53. Подставьте значения в шаблон с плейсхолдерами `${name}`. Неизвестный ключ является ошибкой; ``${name}` должен превращаться в буквальный `${name}`.

```
func RenderTemplate(tpl string, vars map[string]string) (string, error)
```

54. Расширьте шаблонизатор: поддержите `${name:-default}`, вложенные плейсхолдеры в `default` и обнаружение циклических ссылок переменных.

```
func RenderTemplateAdvanced(tpl string, vars map[string]string) (string, error)
```

55. Проверьте баланс `()`, `[]` и `{}`. Скобки внутри одинарных и двойных кавычек игнорируются; обратный слеш экранирует следующий символ внутри кавычки.

```
func ValidateBrackets(s string) error
```

56. Проверьте корректность кавычек и `escape`-последовательностей в строке: поддержите одинарные и двойные кавычки, `\`, `\n`, `\t`, `\r`, `\xNN` и `\uNNNN`.

```
func ValidateQuotedString(s string) error
```

57. Декодируйте строковый литерал в стиле Go без `strconv.Unquote`. Поддержите основные `escape`-последовательности, `hex` и `Unicode`; отклоняйте суррогаты и некорректный UTF-8.

```
func UnquoteString(literal string) (string, error)
```

58. Закодируйте произвольную строку как валидный двойной строковый литерал Go: управляющие символы экранируйте, печатный `Unicode` сохраняйте, кавычки и обратные слешы защищайте.

```
func QuoteString(s string) string
```

59. Найдите длину самой длинной подстроки без повторяющихся `Unicode`-символов за $O(n)$.

```
func LongestUniqueSubstring(s string) int
```

60. Верните самую длинную подстроку, содержащую не более `k` различных `Unicode`-символов. При равной длине выберите самую левую.

```
func LongestSubstringKDistinct(s string, k int) string
```

61. Найдите минимальное окно строки `s`, содержащее все `Unicode`-символы строки `pattern` с учётом кратности. Если окон несколько, верните самое левое.

```
func MinWindow(s, pattern string) (string, bool)
```

62. Верните байтовые индексы всех позиций в `s`, с которых начинается анаграмма `pattern`. Строки корректны в UTF-8, символы `pattern` учитываются с кратностью.

```
func FindAnagramStarts(s, pattern string) []int
```

63. Реализуйте поиск всех вхождений `pattern` алгоритмом Кнута-Морриса-Пратта. Верните байтовые индексы, включая пересекающиеся вхождения.

```
func KMPFindAll(s, pattern string) []int
```

64. Вычислите префикс-функцию для последовательности Unicode-символов: `pi[i]` - длина максимального собственного префикса, совпадающего с суффиксом подстроки `[0..i]`.

```
func PrefixFunction(s string) []int
```

65. Вычислите Z-функцию для последовательности Unicode-символов за $O(n)$.

```
func ZFunction(s string) []int
```

66. Реализуйте сопоставление с wildcard-шаблоном: `?` соответствует одному Unicode-символу, `*` - любой последовательности. Алгоритм не должен иметь экспоненциальную сложность.

```
func WildcardMatch(s, pattern string) bool
```

67. Реализуйте упрощённый `regex`: `.` соответствует одному Unicode-символу, `x*` - нулю или более повторениям предыдущего атома. Вся строка должна совпасть целиком.

```
func RegexMatch(s, pattern string) (bool, error)
```

68. Вычислите расстояние Левенштейна между двумя строками по Unicode-символам. Используйте $O(\min(n, m))$ дополнительной памяти.

```
func Levenshtein(a, b string) int
```

69. Вычислите расстояние Дameraу-Левенштейна, где перестановка двух соседних Unicode-символов стоит одну операцию.

```
func DamerauLevenshtein(a, b string) int
```

70. Найдите одну наибольшую общую подпоследовательность двух строк по Unicode-символам. При нескольких ответах выберите лексикографически минимальный.

```
func LCS(a, b string) string
```

71. Найдите самую длинную палиндромную подстроку за $O(n^2)$ времени или быстрее. При равной длине выберите самую левую.

```
func LongestPalindrome(s string) string
```

72. Подсчитайте все палиндромные подстроки по Unicode-символам. Одинаковый текст на разных позициях считается разными подстроками.

```
func CountPalindromicSubstrings(s string) int
```

73. Верните все разбиения строки на палиндромные подстроки. Порядок вариантов должен быть детерминированным: сначала более короткий первый фрагмент.

```
func PalindromePartitions(s string) [][]string
```

74. Добавьте минимальное число Unicode-символов в начало строки, чтобы получить палиндром, и верните результат. Целевая сложность - $O(n)$.

```
func MakeShortestPalindrome(s string) string
```

75. Найдите минимальное число вставок в произвольные позиции, необходимое для превращения строки в палиндром.

```
func MinInsertionsForPalindrome(s string) int
```

76. Проверьте, можно ли полностью разбить строку на слова из словаря. Одно слово можно использовать многократно.

```
func CanSegment(s string, dict map[string]struct{}) bool
```

77. Верните все предложения, получаемые разбиением строки на слова из словаря. Добавьте memoization и не создавайте дубликаты.

```
func AllSegmentations(s string, dict map[string]struct{}) []string
```

78. Реализуйте структуру Trie со вставкой, точным поиском, проверкой префикса и удалением слова. Удаление не должно ломать другие слова с общим префиксом.

```
type Trie; func (t *Trie) Insert, Contains, HasPrefix, Delete
```

79. На основе Trie реализуйте автодополнение: по префиксу вернуть не более limit слов в лексикографическом порядке. Повторная вставка слова не создаёт дубль.

```
func (t *Trie) Autocomplete(prefix string, limit int) []string
```

80. Постройте автомат Ахо-Корасик для набора непустых шаблонов и найдите все их вхождения в тексте. Верните patternID и байтовый индекс начала; поддержите пересечения.

```
func FindMany(text string, patterns []string) []Match
```

81. Реализуйте Rabin-Karp для поиска всех вхождений подстроки. Хеш-совпадение обязательно подтверждайте сравнением исходных байтов.

```
func RabinKarpAll(s, pattern string) []int
```

82. Проверьте, является ли одна строка циклическим сдвигом другой, не создавая строку $a+a$ целиком в памяти.

```
func IsRotation(a, b string) bool
```


83. Найдите лексикографически минимальный циклический сдвиг строки по Unicode-символам за $O(n)$ или $O(n \log n)$.

```
func MinimalRotation(s string) string
```

84. Найдите минимальный фрагмент, повторением которого можно получить всю строку. Если строка непериодична, верните её целиком.

```
func SmallestPeriod(s string) string
```

85. Для каждой позиции строки вычислите длину минимального периода префикса, заканчивающегося в этой позиции.

```
func PrefixPeriods(s string) []int
```

86. Сравните версии вида 1.10.0-alpha.2 по правилам SemVer 2.0.0: числовые компоненты, prerelease-идентификаторы и отсутствие prerelease. Некорректные версии отклоняйте.

```
func CompareSemVer(a, b string) (int, error)
```

87. Реализуйте natural comparison: "file9" < "file10". Числовые блоки сравнивайте без преобразования в int, чтобы поддержать числа любой длины; задайте правило для ведущих нулей.

```
func NaturalCompare(a, b string) int
```

88. Отсортируйте строки без учёта регистра, но детерминированно: сначала сравнение в Unicode lower-case, затем исходная строка как tie-breaker. Не меняйте входной слайс.

```
func SortFolded(items []string) []string
```

89. Постройте построчный diff двух текстов на основе LCS. Для каждой строки верните операцию Equal, Insert или Delete и сам текст строки.

```
func DiffLines(oldText, newText string) []DiffOp
```

90. Примените результат предыдущей задачи к исходному тексту. Проверьте, что операции Equal/Delete действительно совпадают с текущими строками; при конфликте верните ошибку.

```
func ApplyDiff(oldText string, ops []DiffOp) (string, error)
```

91. Перенесите текст по словам на строки шириной не более width Unicode-символов. Длинные слова разбивайте, пустые абзацы сохраняйте, завершающие пробелы не добавляйте.

```
func WrapText(s string, width int) ([]string, error)
```

92. Усечь строку так, чтобы результат занимал не более maxBytes байт и оставался корректным UTF-8. Если было усечение, по возможности добавьте suffix, также не нарушив лимит.

```
func TruncateUTF8(s string, maxBytes int, suffix string) string
```

93. Разбейте строку на части максимум по `maxRunes` Unicode-символов. Не разрывайте UTF-8 последовательности; при `maxRunes <= 0` верните ошибку.

```
func ChunkByRunes(s string, maxRunes int) ([]string, error)
```

94. Разбейте корректную UTF-8 строку на минимальное число частей, каждая из которых занимает не более `maxBytes` байт. Один Unicode-символ нельзя разрывать; невозможный лимит должен давать ошибку.

```
func ChunkByBytes(s string, maxBytes int) ([]string, error)
```

95. Исправьте произвольную последовательность байтов: каждую некорректную UTF-8 последовательность замените ровно одним `RuneError` и верните число замен.

```
func SanitizeUTF8(data []byte) (string, int)
```

96. Сравните две строки после Unicode-нормализации NFC. Разрешается использовать golang.org/x/text/unicode/norm; добавьте тесты для составного и комбинированного написания é.

```
func EqualNFC(a, b string) bool
```

97. Реализуйте регистронезависимый поиск Unicode-подстроки с возвратом байтового индекса в исходной строке. Учтите, что case folding может менять длину представления.

```
func IndexFold(s, sub string) int
```

98. Разверните строку по пользовательским символам (grapheme clusters), не отделяя комбинируемые знаки и модификаторы emoji от базового символа. Разрешается подключить библиотеку сегментации Unicode.

```
func ReverseGraphemes(s string) string
```

99. Напишите потоковый счётчик слов из `io.Reader`. Чтение идёт блоками произвольного размера, UTF-8 символ и слово могут пересекать границу блоков; весь текст нельзя загружать в память.

```
func CountWordsReader(r io.Reader) (int64, error)
```

100. Напишите потоковый поиск всех вхождений UTF-8 шаблона в `io.Reader`. Совпадение может пересекать границы чтения; верните байтовые смещения и используйте память `O(len(pattern))`.

```
func FindAllReader(r io.Reader, pattern string) ([]int64, error)
```